Lab Sheet 07

Title: Temperature-Based Servo Motor Control with Manual Override using ESP32 & Blynk.

Aims:

To design and implement a temperature-based servo motor control system using the ESP32 microcontroller, where the servo motor operates automatically according to temperature variations, while also providing a manual override feature through the Blynk mobile application for real-time user control and monitoring.

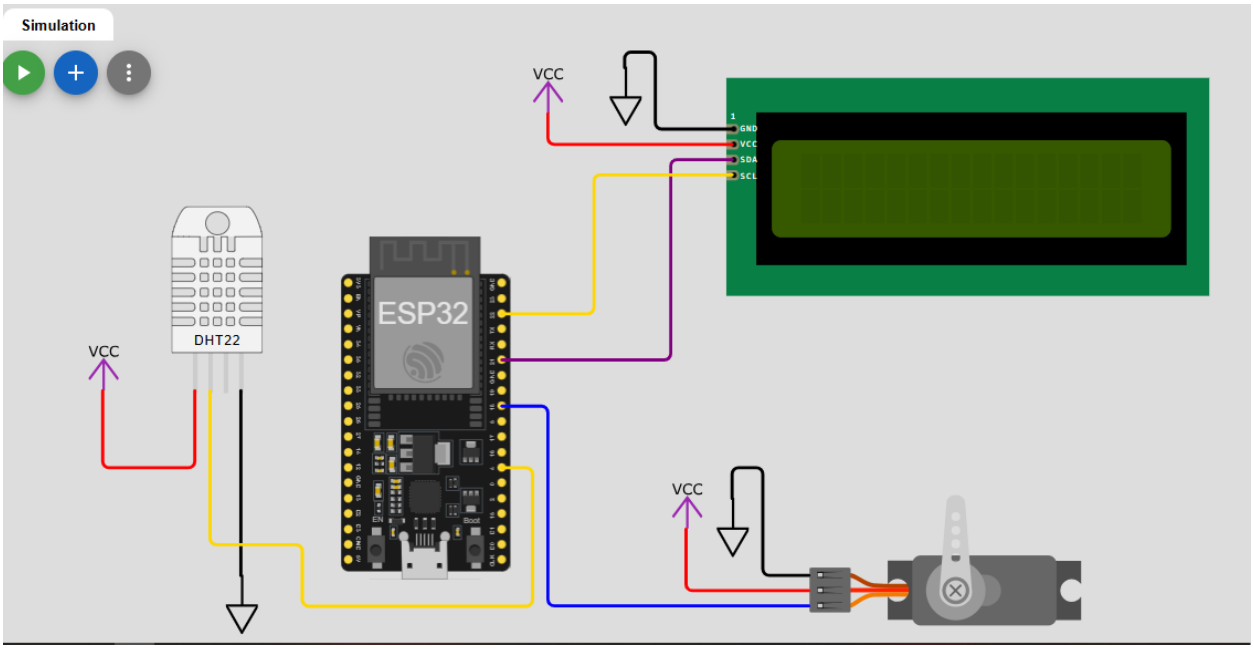
Blynk IoT Platform

Blynk is a cloud-based IoT platform that enables real-time bi-directional communication between microcontrollers and smartphone apps. It uses virtual pins (V0–V255) as data channels. The Blynk library abstracts WebSocket/HTTP calls, allowing the developer to read/write virtual pins with simple function calls.

https://blynk.io

Task 01:

1. Project Setup in Wokwi / VS Code
2. Circuit Design
    a. Connect the DHT sensor to an ESP32 GPIO pin
    b. Connect the servo motor to a PWM-capable GPIO pin
    c. Connect the LCD (I2C) to the ESP32.



3. Library Integration

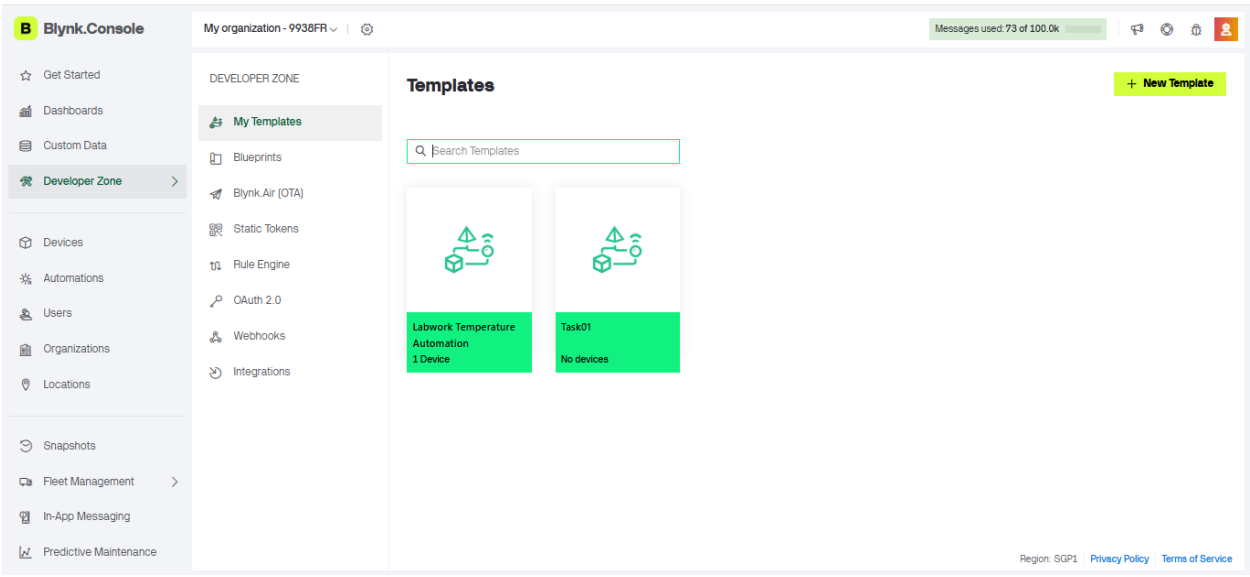
I. Blynk
II. DHT sensor library
III. Servo
IV. LiquidCrystal_I2C

```
[env:esp32dev]
platform = espressif32
board = esp32dev
framework = arduino
lib_deps =
    blynkkk/Blynk@^1.3.2
    adafruit/DHT sensor library@^1.4.7
    madhephaestus/ESP32Servo@^3.2.0
    marcoschwartz/LiquidCrystal_I2C@^1.1.4
```

4. Blynk Configuration

a.Create a New Template in Blynk (IoT platform)

Sidebar → Developer Zone → My Templates → Create New Template → Enter Template Name → Select Hardware → Choose Connection Type → Click Create → Add Datastreams → Design Dashboard → Save



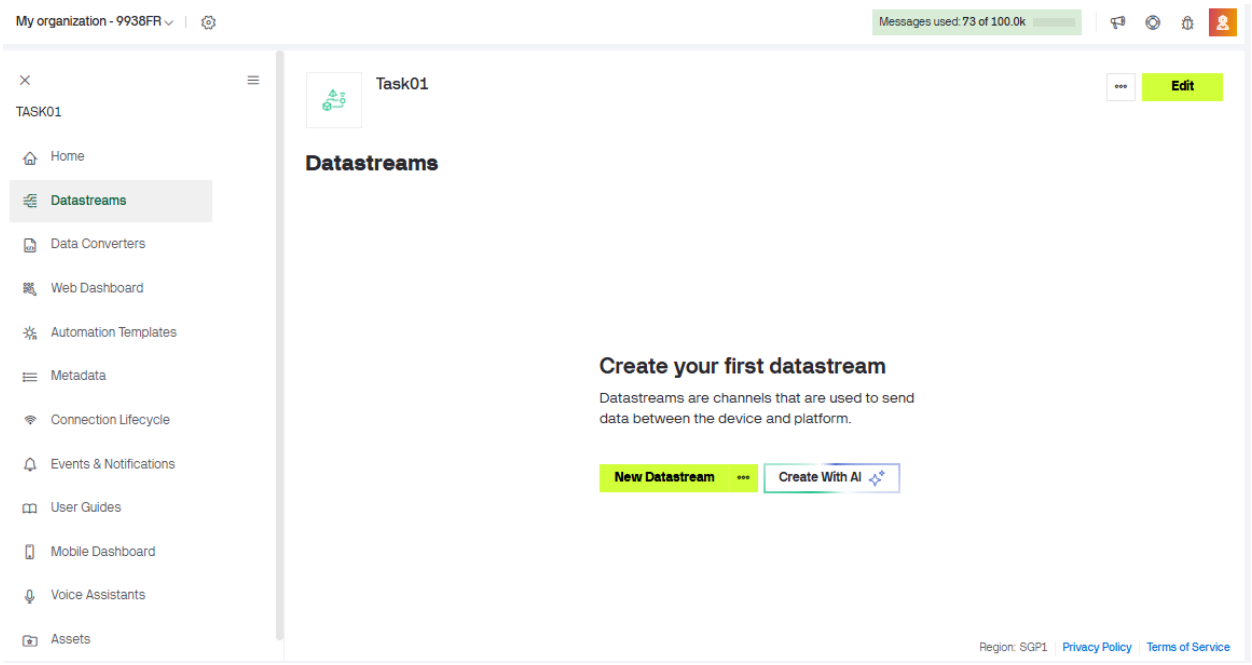
b.Assign virtual pins

I. Temperature: Units: Degrees

II. iHumidity: Units: Percentage, % | Max: 100

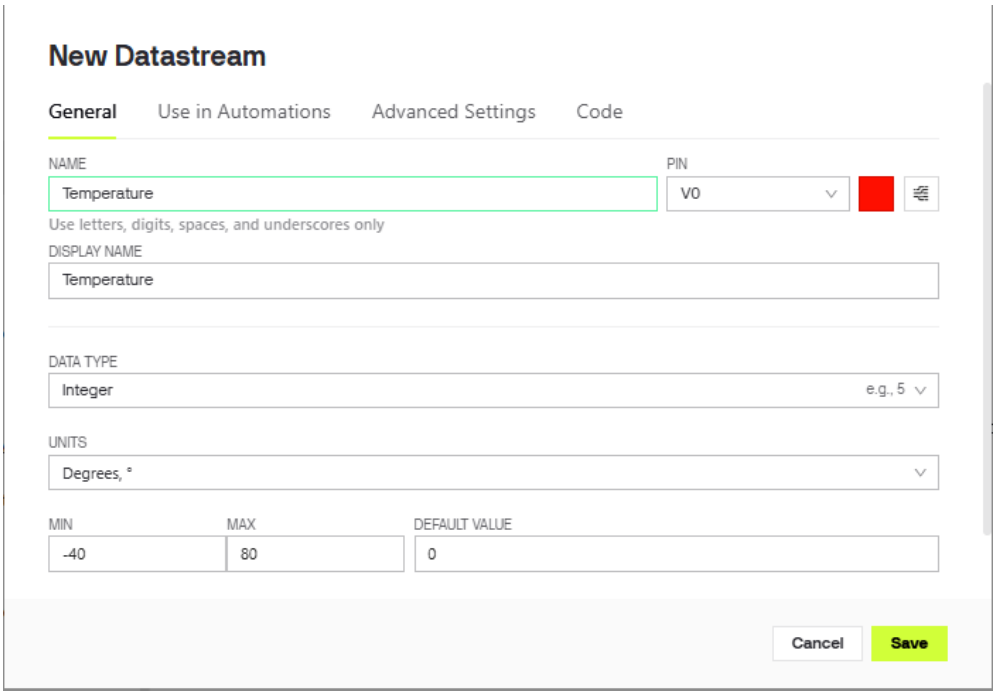
III. Servo: Units: Degrees, ° | Max: 180

Datastreams → Add New Datastream → Virtual Pin



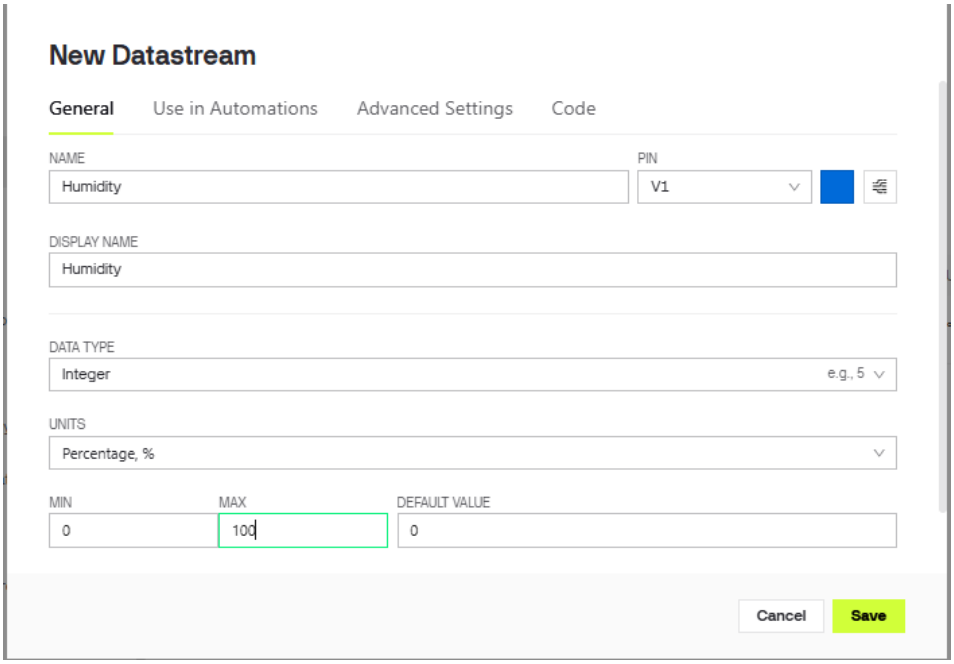
🌡 Temperature

- Virtual Pin → V0
- Name → Temperature
- Data Type → Double / Float
- Units → °C (Degrees)
- Min → -40 (optional)
- Max → 80 (optional)



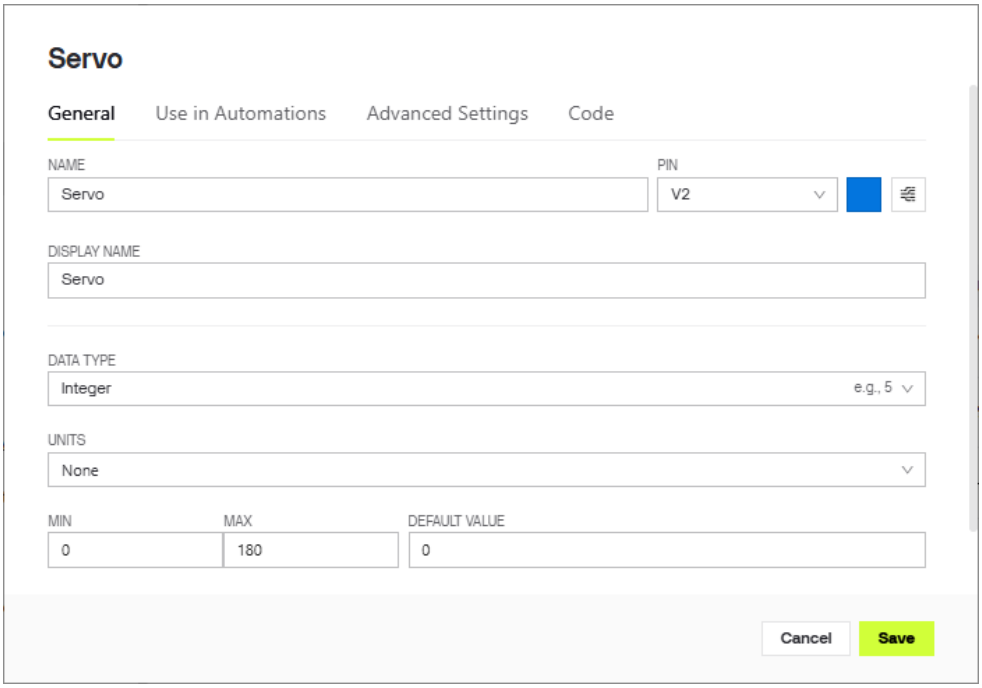
💧 Humidity (iHumidity)

- Virtual Pin → V1
- Name → Humidity
- Data Type → Integer / Double
- Units → % (Percentage)
- Min → 0
- Max → 100



🎛️ Servo Motor

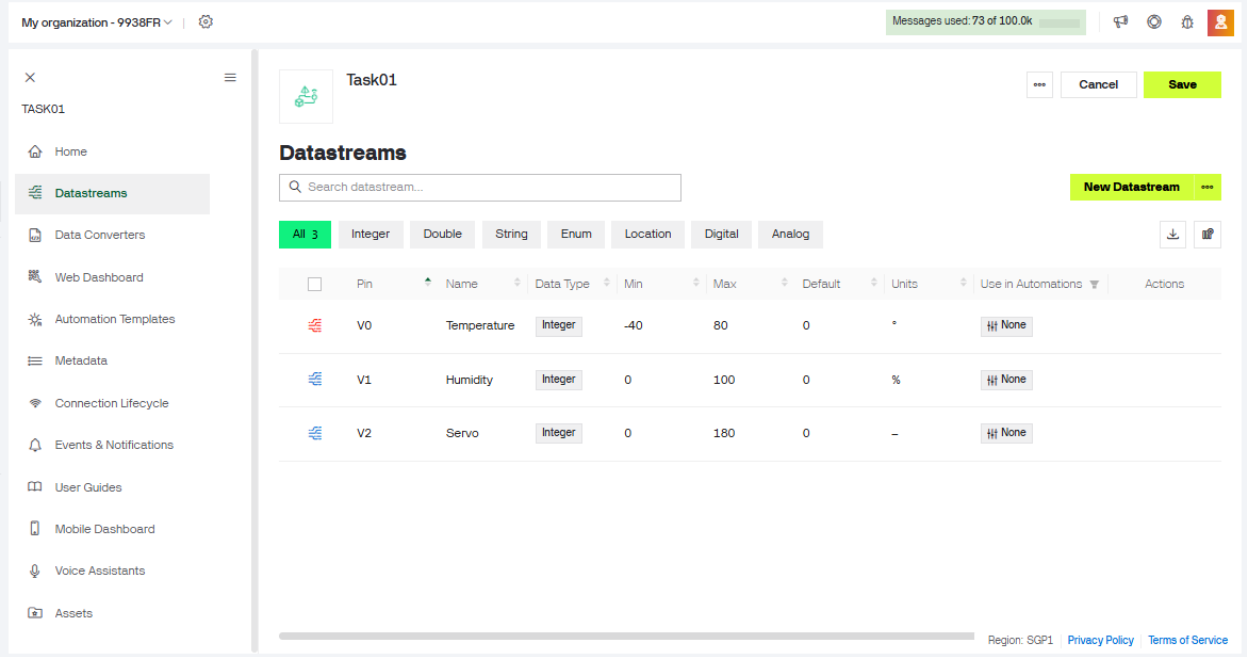
- Virtual Pin → V2
- Name → Servo
- Data Type → Integer
- Units → ° (Degrees)
- Min → 0
- Max → 180



🔁 Final Summary (Quick View)

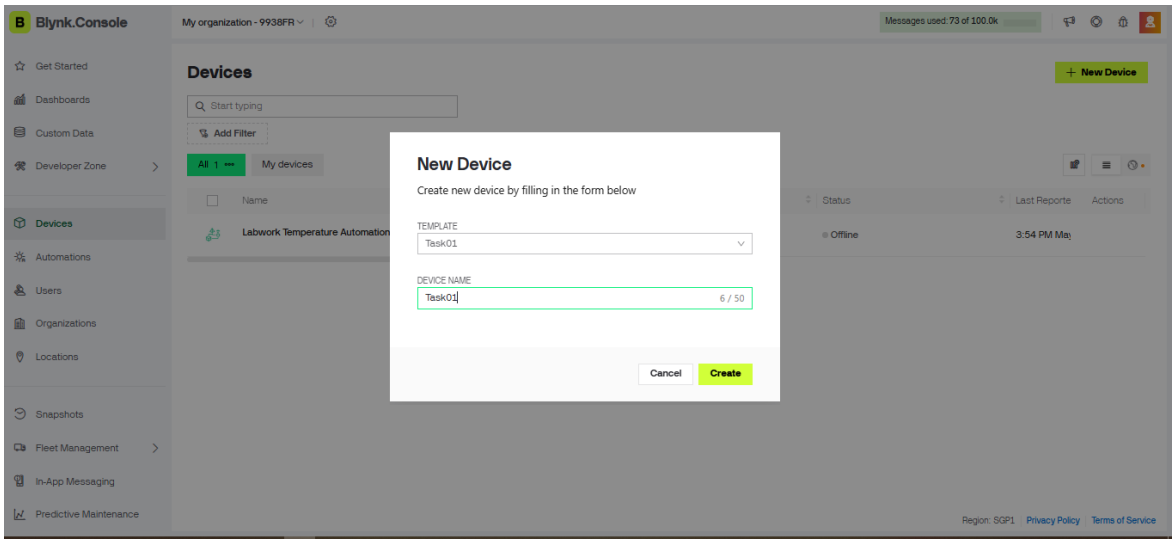
Datastreams → Add →

- V0 → Temperature (°C)
- V1 → Humidity (%)
- V2 → Servo (0–180°)



c.Create a New Device

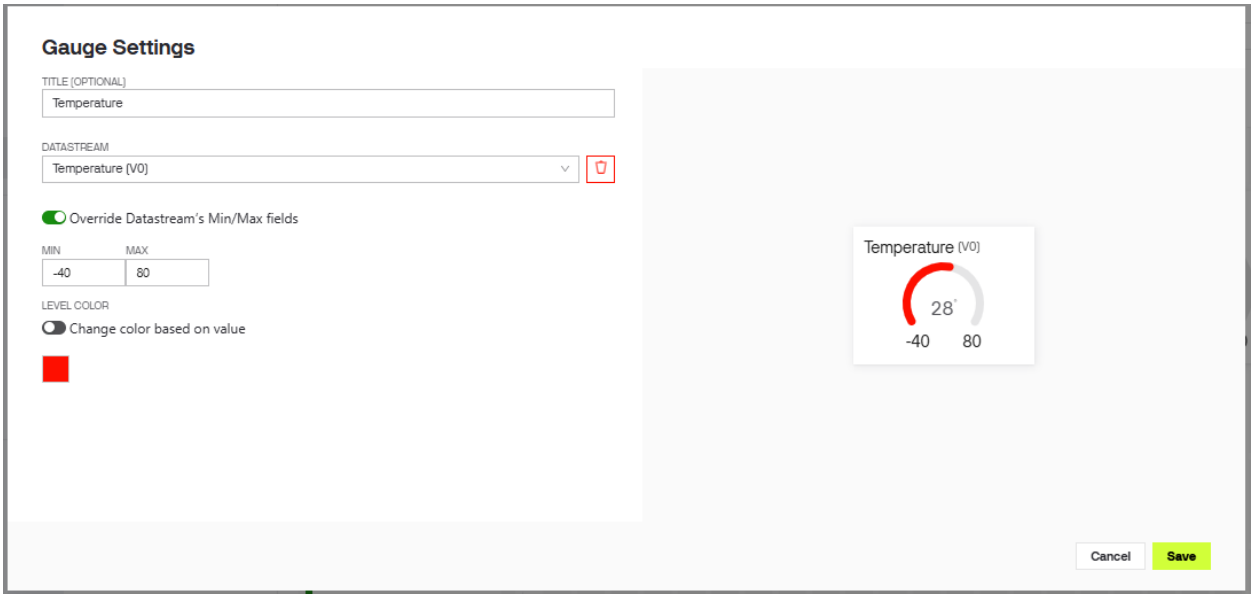
Sidebar → Developer Zone → My Devices → + New Device → From Template → Select Template → Enter Device Name → Create Device → Get Auth Token (or Device Credentials) → Save

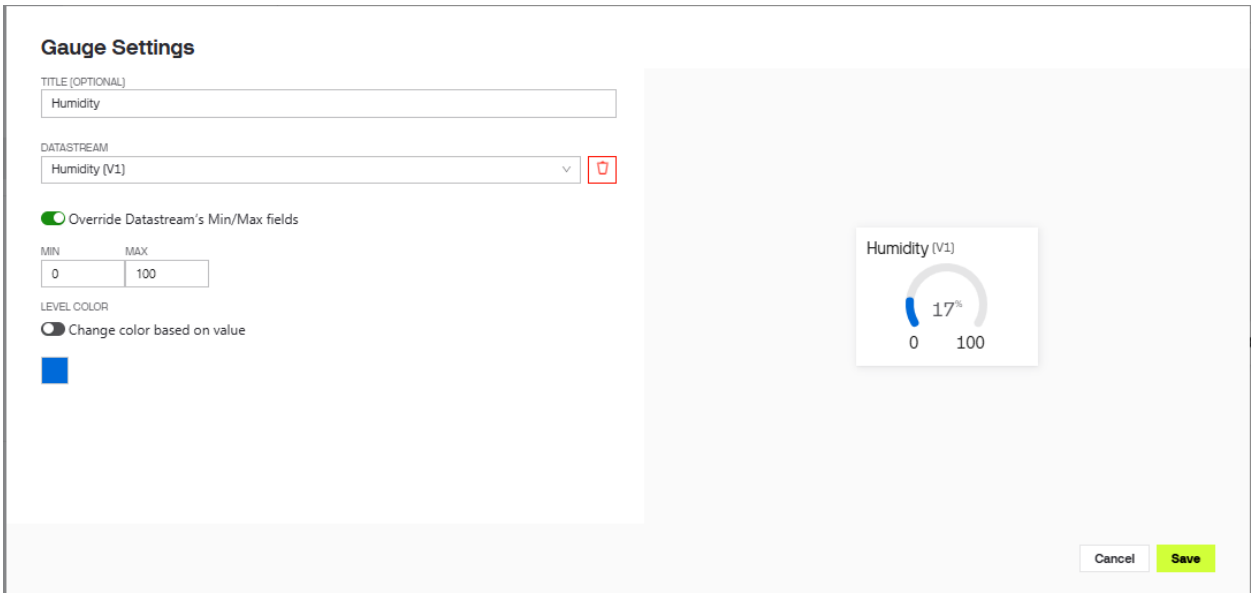


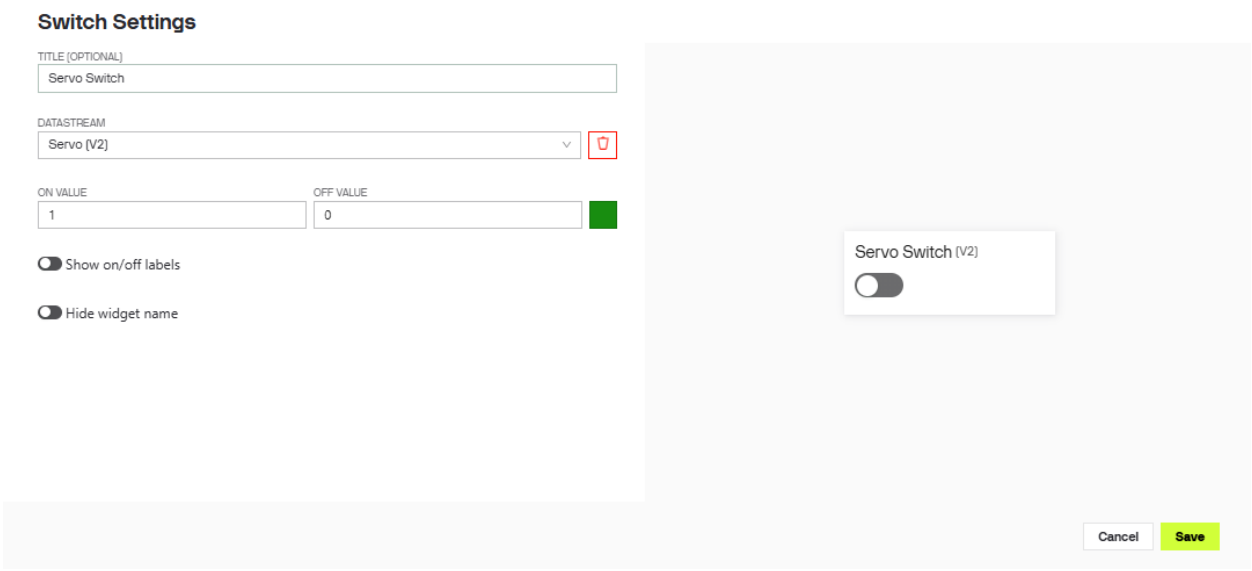
d.Add widgets:

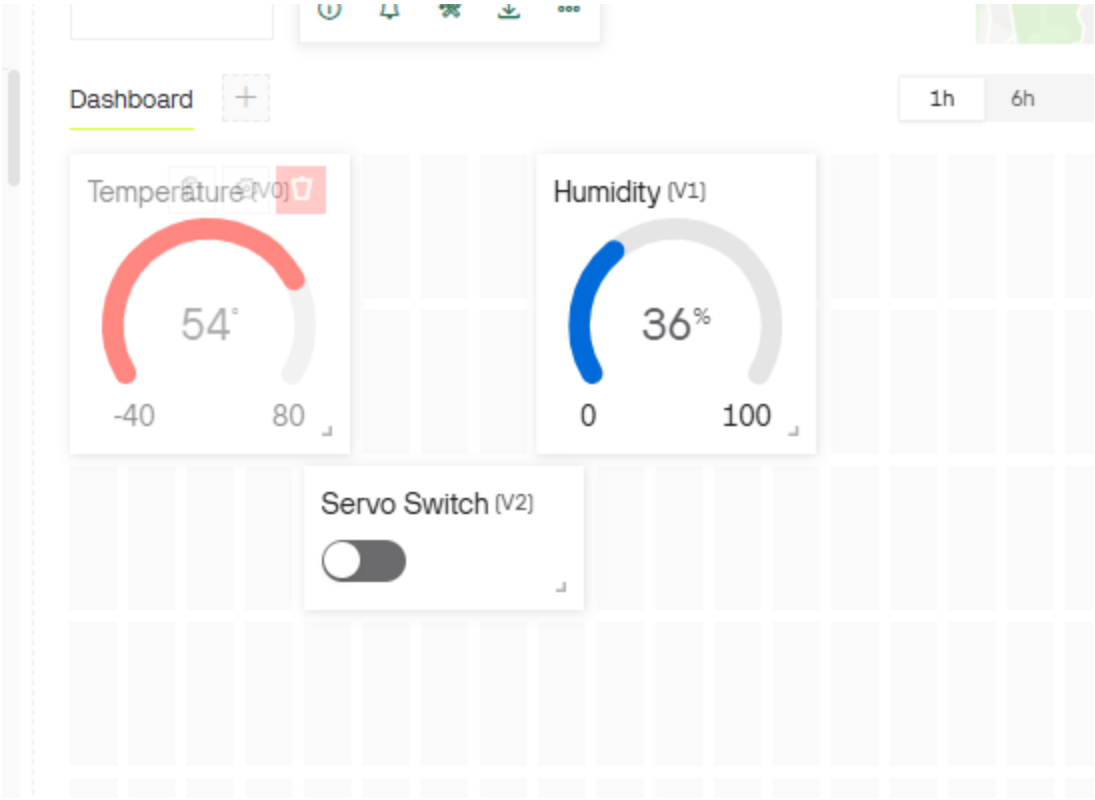
I. **Temperature display (Gauge)**
II. **Humidity display (Gauge)**
III. **Servo (Switch)**

My Device → Open Dashboard → Edit Mode → + Add Widget → Gauge → Select Temperature (V0) → Set Units °C → Save → Add Widget → Gauge → Select Humidity (V1) → Set Units % → Save → Add Widget → Switch (or Slider) → Select Servo (V2) → Set Range 0–180° → Save









e.Obtain Template ID, Template Name and Auth Token

```
#define BLYNK_TEMPLATE_ID "TMPL6uxMnzyf7"
#define BLYNK_TEMPLATE_NAME "Task01"
```

```
#define BLYNK_AUTH_TOKEN
"Jmv5Dwj6JzoPuWPqhYbtjjmdcLusNYy9"
```

5. Programming Logic Development in Wokwi / VS Code

    a. Initialize Wi-Fi and Blynk connection

    b. Read temperature and humidity from DHT sensor

    c. Display the values in LCD

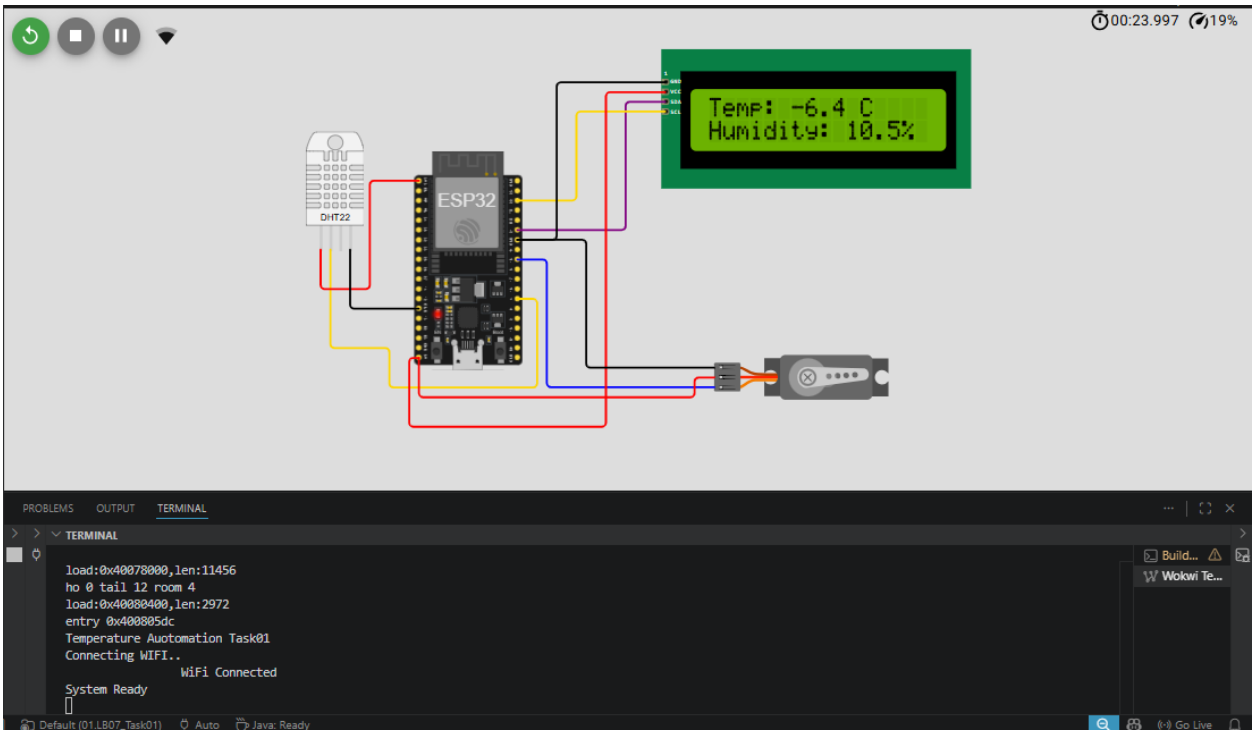
```
#define BLYNK_TEMPLATE_ID "TMPL6uxMnzyf7"
#define BLYNK_TEMPLATE_NAME "Task01"
#define BLYNK_AUTH_TOKEN "Jmv5Dwj6JzoPuWPqhYbtjjmdcLusNYy9"

#include <Arduino.h>
#include <WiFi.h>
#include <WiFiClient.h>
#include <BlynkSimpleEsp32.h>
#include <DHT.h>
#include <ESP32Servo.h> // Fixed: Standard Servo.h doesn't work on ESP32, changed
to ESP32Servo
#include <LiquidCrystal_I2C.h>
#include <Wire.h>

#define DHTPIN 4
#define DHTTYPE DHT22
#define SERVO_PIN 18
#define VPIN_TEMP V0
#define VPIN_HUM V1
#define VPIN_SERVO V2

char ssid[] = "Wokwi-GUEST";
char pass[] = "";

DHT dht(DHTPIN, DHTTYPE);
LiquidCrystal_I2C lcd(0x27, 16, 2);
Servo myservo;

// Added tracker to replace bad delay(2000) in loop
unsigned long lastTime = 0;
```

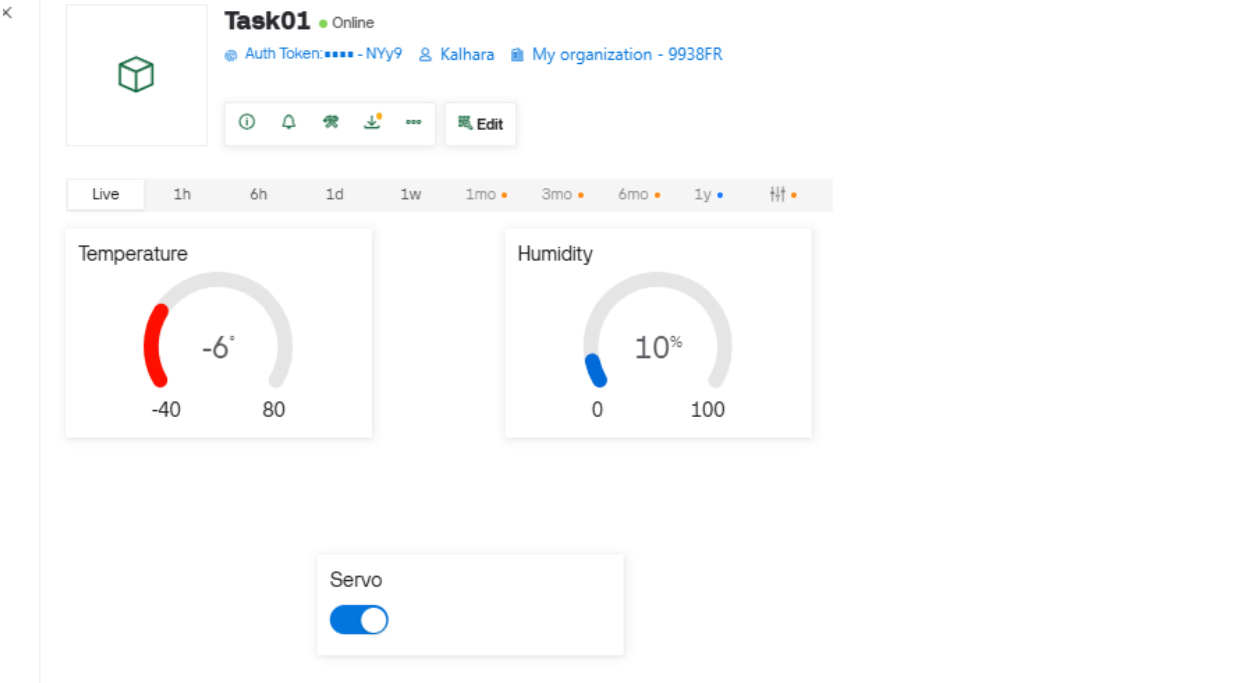
```
BLYNK_WRITE(VPIN_SERVO)
{
  int buttonState = param.asInt();

  if (buttonState == 1)
  {
    myservo.write(90);
  }
  else
  {
    myservo.write(0);
  }
}

void setup()
{
  Serial.begin(115200);
  Serial.println("Temperature Auotomation Task01");

  lcd.init();
  lcd.backlight();
  lcd.clear();

  // Fixed: WiFi.begin must be called BEFORE checking status loop
  WiFi.begin(ssid, pass);

  lcd.setCursor(0, 0);
  lcd.print("Connecting WIFI");
  Serial.print("Connecting WIFI");

  int attempt = 0;
  while (WiFi.status() != WL_CONNECTED && attempt < 20)
  {
    delay(500);
    lcd.print(".");
    Serial.print(".");
    attempt++;
  }

  lcd.clear();
  if (WiFi.status() == WL_CONNECTED)
  {
    lcd.setCursor(0, 0);
    lcd.println("WiFi Connected");
    Serial.println("\nWiFi Connected");
```

```
  }
  else
  {
    lcd.setCursor(0, 0);
    lcd.println("WiFi Conn Failed");
    Serial.println("WiFi Connection Failed");
  }

  delay(1500);
  lcd.clear();

  Blynk.begin(BLYNK_AUTH_TOKEN, ssid, pass);

  dht.begin();

  // Fixed: ESP32 PWM timer allocation for servo stability
  ESP32PWM::allocateTimer(0);
  myservo.setPeriodHertz(50);

  myservo.attach(SERVO_PIN, 500, 2400);
  myservo.write(0);

  lcd.setCursor(0, 0);
  lcd.print("System Ready");
  Serial.println("System Ready");
  delay(1000);
  lcd.clear();
}

void loop()
{
  Blynk.run();

  // Fixed: Using millis() instead of delay(2000) to keep Blynk from
disconnecting
  if (millis() - lastTime >= 500)
  {
    lastTime = millis();

    float temperature = dht.readTemperature();
    float humidity = dht.readHumidity();

    if (isnan(temperature) || isnan(humidity))
    {
      lcd.clear();
```

```
      lcd.setCursor(0, 0);
      lcd.print("Sensor Error");
      Serial.println("Temperature Or Humidity Read Error !");
      return;
    }

    lcd.clear();
    lcd.setCursor(0, 0);
    lcd.print("Temp: ");
    lcd.print(temperature, 1);
    lcd.print(" C");

    lcd.setCursor(0, 1);
    lcd.print("Humidity: ");
    lcd.print(humidity, 1);
    lcd.print("%");

    Blynk.virtualWrite(VPIN_TEMP, temperature);
    Blynk.virtualWrite(VPIN_HUM, humidity);
  }
}
```

6. Simulation & Testing in Wokwi / VS Code
    a. Run simulation
    b. Adjust temperature values in Wokwi / VS Code
    c. Observe servo movement
    d. Verify Blynk interaction


00:23.997
19%
Temp: -6.4 C
Humidity: 10.5%
DHT22
ESP32
PROBLEMS
OUTPUT
TERMINAL
TERMINAL
load:0x40078000,len:11456
ho 0 tail 12 room 4
load:0x40080400,len:2972
entry 0x400805dc
Temperature Auotomation Task01
Connecting WIFI..
WiFi Connected
System Ready
Build...
Wokwi Te...


Task01
Online
Auth Token:▪▪▪▪ - NYy9
Kalhara
My organization - 9938FR
Edit
Live
1h
6h
1d
1w
1mo
3mo
6mo
1y
Temperature
-6°
-40
80
Humidity
10%
0
100
Servo